

Bank Licensing and Compliance Portal

Design Document

Date: 11th May 2026

1. Problem and goals

The National Bank of Rwanda licenses every commercial, development, cooperative, and mortgage bank operating in Rwanda. Today, that process is manual: applications arrive by email, supporting documents are tracked in spreadsheets, and approval decisions move through informal channels. There is no single source of truth, no reliable audit trail, and no visibility into where any application currently stands.

This portal replaces that process end-to-end. It receives applications, holds the documents, runs them through a defined review workflow with strict separation of duties, captures an immutable audit trail, and gives the regulator real-time visibility into the pipeline. The system is grounded in BNR Regulation 2310-13 of 2018 and encodes its specific facts (categories, capital tiers, document checklist, statutory time clocks, fees, decision types) as first-class entities rather than free-text gestures.

1.1 Goals

- Authenticated access for applicants, reviewers, decision-makers, and administrators.
- A regulation-grounded state machine from submission to in-principle decision, enforced server-side.
- Versioned document handling with server-enforced size and type checks and at-rest encryption.
- Append-only audit trail with hash chain, suitable for evidentiary use.
- Statutory SLA tracking for the 10-working-day completeness window, the 1-month assessment, and the 3-month absolute decision deadline.
- A web client that renders the appropriate surface for each role, also packaged as an Electron desktop application for the regulator.

1.2 Non-goals (this build)

- Post-grant operationalisation: deposit certificate, operating manuals, pre-opening inspection. State-machine seam reserved; full automation deferred.
- Representative-office workflow for foreign banks. Out of scope.
- Annual supervisory update obligations (Article 31). Belongs to a separate supervisory portal.
- Email and SMS transports. Notifications are in-app only; the subsystem is interface-shaped so transports can be added later.
- Cloud-hosted document storage. Simulated locally with envelope encryption behind an interface that swaps to S3 cleanly.

2. Regulation grounding (BNR 2310-13 of 2018)

The regulation's specific facts are encoded directly. Categories drive the capital threshold check. Holdings drive the shareholder model. Specific mandates drive the senior-manager qualification check. Defined timeframes drive the statutory clock. The regulation also drives the four decision outcomes and the fee table.

2.1 Bank categories and minimum paid-up capital

Category	Code	Minimum paid-up capital (RWF)
Commercial Bank	COMMERCIAL	20,000,000,000
Development Bank	DEVELOPMENT	50,000,000,000
Cooperative Bank	COOPERATIVE	10,000,000,000
Mortgage Bank	MORTGAGE	10,000,000,000

2.2 Statutory SLAs

Clock	Trigger	Deadline	Action on breach
Completeness letter	Application submitted	10 working days	A letter of information must be issued.
Assessment	File marked complete	1 month	Decision communicated; otherwise delay notice is required.
Delay notice	Assessment elapsed without a decision	15 calendar days	Notice of delay reasons sent to the applicant.
Absolute decision	File marked complete	3 months	The final decision must be issued.
Licence fee	Decision to grant notified	1 month	Applicant must pay RWF 5,000,000.

2.3 Decision types

- Grant: licence issued without conditions.
- Grant with conditions: licence issued subject to a structured list of conditions, each with a target satisfaction date.
- Grant limited licence: a licence issued for a defined subset of the category's permitted activities.
- Refuse: application refused with structured reasons, each citing the failing article.

2.4 Fees

Fee	Amount (RWF)	Trigger	Deadline
Application fee	2,000,000	Submission	Must accompany the application; non-refundable.
Licence fee	5,000,000	Grant notification	Within 1 month after notification.
Annual supervision	0.5% of gross income (min 5,000,000)	End of financial year	Within 4 months after FY end (deferred).

3. Architecture

The system is a single-page Angular 18 web application backed by a NestJS HTTP API and a PostgreSQL database. The same Angular bundle is packaged into an Electron desktop shell for the regulator. The API is the only place that talks to the database and the document store; the client is treated as untrusted.

3.1 Components

- API service (NestJS, Fastify adapter). Owns authentication, authorisation, the state machine, audit writes, document metadata, regulation enforcement, and validation.
- Database (PostgreSQL 16). Holds users, applications, capital declarations, significant shareholders, senior managers, document metadata, decisions, letters of information, SLA clocks, fees, audit entries, notifications, and refresh tokens.
- Document store. Encrypted local disk in this build (AES-256-GCM with envelope-wrapped per-document DEKs). Interface-shaped, so S3 or object storage swaps in cleanly.
- Web client (Angular 18, standalone components, Tailwind + Material). Communicates with the API over HTTPS. Renders the appropriate surface for the user's role. Holds no regulation rules locally.
- Desktop shell (Electron). Hardened defaults: contextIsolation, sandbox, no nodeIntegration, signed updates, DevTools disabled in production.
- Scheduled jobs. SLA watcher (every 15 minutes) evaluates clocks and emits warnings and breaches. Audit snapshot (daily) computes a Merkle root and signs it; the WORM destination is deferred.

3.2 Why this shape

Centralising every rule in the API is the only way to honour the requirement that frontend bypass cannot grant access. The API is the single boundary that defines what a user can do, what state changes are legal, and what gets recorded. The web app is a reader of the regulation, not its enforcer; every threshold, every minimum, every required-slot list comes from an API response.

See ``docs/architecture.drawio`` for the layered view.

4. State machine

State	Meaning	Who acts next
DRAFT	Owned by the applicant; not yet visible to the regulator.	Applicant (submit/withdraw)
SUBMITTED	In the regulator queue, no reviewer assigned.	Reviewer (claim) / Admin (assign)
UNDER_REVIEW	A reviewer is examining the application.	Reviewer (request info / recommend)

State	Meaning	Who acts next
AWAITING_APPLICANT_RESPONSE	The reviewer has requested additional information.	Applicant (resubmit/withdraw)
RECOMMENDED_FOR_APPROVAL	The reviewer has finished and recommends approval.	Approver (decide)
RECOMMENDED_FOR_REJECTION	The reviewer has finished and recommends rejection.	Approver (decide)
APPROVED	Final decision recorded (grant / conditional/limited).	Terminal
REJECTED	Final decision recorded (refuse).	Terminal
WITHDRAWN	Applicant cancelled before any decision.	Terminal

Illegal transitions are rejected at the API with ``409 ILLEGAL_TRANSITION``. Terminal states have no outgoing transitions. The reviewer of an application cannot record the final decision on the same application; the ``SeparationOfDutiesGuard`` enforces this server-side.

Concurrent transitions are handled by optimistic locking on ``applications.row_version`` inside a single ``SELECT ... FOR UPDATE`` transaction. One writer wins; the other receives ``409 VERSION_CONFLICT``; both events appear in the audit log.

5. Data model

Schema highlights only; see ``docs/erd.drawio`` for the full diagram. Foreign keys carry ``ON DELETE RESTRICT``. Timestamps are ``TIMESTAMPTZ`` and server-generated.

5.1 Core entities

- `users` — identity and role (APPLICANT, REVIEWER, APPROVER, ADMIN). Argon2id password hash. MFA secrets and Argon2-hashed recovery codes in a 1:1 child table.
- `applications` — application kind (NEW_BANK, FOREIGN_SUBSIDIARY), bank category enum, reference number, applicant FK, state enum, current reviewer, approver, `row_version` (optimistic lock), submission and decision timestamps.
- `capital_declarations` — 1:1 with applications. `amount_rwf` BIGINT, source summary, `attested_at`, `attested_by_user_id`.
- `significant_shareholders` — type (INDIVIDUAL, CORPORATE), `holding_percentage` ≥5.00, `source_of_funds`, `fit_and_proper_status` (PENDING / CLEARED / FAILED).
- `senior_managers` — role (CEO, CFO, COO, CIA, CRO, CIO, OTHER), qualifications, certifications, `years_experience`, `fit_and_proper_status`.

- `application_documents` — per-slot versioning; uploader; AES-256-GCM ciphertext on disk; per-document DEK wrapped by a KEK. Optional `shareholder_id` or `manager_id` when the slot is per-owner.
- `decisions` — type (GRANT, GRANT_WITH_CONDITIONS, GRANT_LIMITED, REFUSE), conditions JSONB, `allowed_activities`, reasons JSONB, `justification`, `decided_by`, `decided_at`.
- `letters_of_information` — type (COMPLETE, DEFICIENT), structured deficiencies, `rectification_deadline`, signed PDF document FK.
- `application_sla_clocks` — kind, `started_at`, `due_at`, `warned_at`, `breached_at`, `closed_at`.
- `fees` — kind (APPLICATION, LICENCE, SUPERVISION), `amount_rwf`, `due_at`, `paid_at`, `evidence_document_id`.
- `application_audit` — append-only. DB role grants INSERT/SELECT only; a BEFORE UPDATE OR DELETE OR TRUNCATE trigger raises unconditionally; every row hash-chains to the previous entry per application using RFC 8785 canonicalisation.
- `refresh_tokens` — hashed at rest; single-use rotation; family + device binding; reuse detection revokes the whole family.

5.2 Reference tables

- `bank_category_thresholds`: seeded from the regulation's capital requirements. Read-only at runtime.
- `Senior_manager_role_minimums`: seeded from statutory qualification standards. Required certifications and minimum years per role.
- `Document_slot_specs`: seeded from the mandated document checklist. Required-for-categories, allowed MIME types, max bytes, per-owner scope.

6. User journeys

Three primary journeys describe the end-to-end flow. Each row is one user-visible step; each step pairs the user action with the system response so a reviewer can trace it through the implementation.

6.1 Applicant: submit a licence application

Step	Applicant action	System response
1	Login (no MFA in v1 for applicants).	JWT access token + refresh token issued; refresh stored in sessionStorage.
2	Click "New application"; pick bank category.	Draft created; capital threshold for the chosen category displayed.
3	Fill identity, contact, summary.	Form-builder validates client-side; API revalidates on save.
4	Declare paid-up capital (RWF).	API enforces the amount $\geq$ threshold(category) and writes a structured finding otherwise.

Step	Applicant action	System response
5	Add significant shareholders (≥5% holdings each).	Holdings sum is validated; per-shareholder fit-and-proper status defaults to PENDING.
6	Add senior managers (CEO, CFO, COO, CIA, CRO, CIO).	Role-keyed qualification minimums shown as hints; API enforces on save.
7	Upload supporting documents to the slots required for the category.	Streaming upload capped at 5 MB; MIME sniffed; per-slot versioning atomic; ciphertext on disk.
8	Pay the application fee (RWF 2,000,000) and upload proof.	Fee row marked PAID; absence blocks state transition out of DRAFT.
9	Review compliance findings panel; resolve blocking items.	Blocking count must reach zero before submit.
10	Submit.	State → SUBMITTED. Reference number issued. Completeness SLA clock starts (10 working days).
11	Receive a letter of information (DEFICIENT) listing rectifications.	State → AWAITING_APPLICANT_RESPONSE. PDF download available. Rectification deadline visible.
12	Upload missing documents; resubmit.	State → UNDER_REVIEW. Assigned reviewer notified.
13	Receive decision letter.	State → APPROVED or REJECTED. Decision detail, conditions, or reasons visible. Licence fee (if granted) added to fees panel with 1-month deadline.

6.2 Reviewer: review and recommend

Step	Reviewer action	System response
1	Login with MFA (TOTP or recovery code).	Two-step flow; refresh token bound to device.
2	Open the review queue.	Paginated list of SUBMITTED applications; server-side search and filter.
3	Claim an application.	Optimistic lock on current_reviewer_id; one winner; state → UNDER_REVIEW.
4	Open the application detail.	Compliance findings, SLA clocks, documents, shareholders, managers, capital, fees rendered from the API.
5	If completeness clock close to breach, issue COMPLETE letter; or if deficiencies, issue DEFICIENT letter with structured deficiency rows and rectification deadline.	PDF rendered, signed by the snapshot signer, attached as a document, notification sent to the applicant. Completeness clock closes; assessment + absolute clocks start.
6	Mark shareholder fit-and-proper CLEARED or FAILED with justification.	Status persisted; audit row written; compliance findings recomputed.
7	Request additional information.	State → AWAITING_APPLICANT_RESPONSE; applicant notified.
8	Recommend approval or rejection with mandatory justification.	Blocked while blocking-count > 0. On submit, state → RECOMMENDED_FOR_APPROVAL or

Step	Reviewer action	System response
		RECOMMENDED_FOR_REJECTION. Approver queue updated.

6.3 Approver: record the final decision

Step	Approver action	System response
1	Login with MFA.	Two-step flow.
2	Open the approver queue.	Multi-state filter limited to RECOMMENDED_FOR_APPROVAL and RECOMMENDED_FOR_REJECTION; paginated.
3	Open the application detail; read the reviewer's recommendation, the compliance findings, the document set, and the audit timeline.	All panels render. "Verify audit chain" available to APPROVER and ADMIN.
4	Click "Record decision". Pick one of four tabs.	Modal opens: Grant / Grant with conditions / Grant limited / Refuse.
5	Grant with conditions: add structured conditions with target satisfaction dates and mandatory justification. Grant limited: pick permitted activities. Refuse: add structured reasons each citing the failing article.	Form validates; submit disabled while invalid.
6	Submit.	SeparationOfDutiesGuard rejects if the approver was a prior reviewer. State → APPROVED or REJECTED. Decision row written. Audit entry written in the same transaction. Applicant notified. Licence fee row (if granted) created with 1-month deadline.

7. Security posture

- Argon2id password hashing, parameter-versioned, upgraded on next successful login.
- Two-step login for internal roles (Reviewer, Approver, Admin) with TOTP MFA and Argon2-hashed single-use recovery codes.
- RS256 JWT access tokens with `kid` rotation. Asymmetric only; HS256 explicitly rejected.
- Refresh tokens: single-use rotation, hashed at rest, family + device binding, reuse detection revokes the whole family.
- Access tokens in memory only on the web; `safeStorage` on Electron. Refresh on `sessionStorage` (web) — closing the tab forces re-login.
- Authorisation failures return 403 (never 404). Stack traces never appear in HTTP responses.
- Login is rate-limited per account and per IP; account lockout after a configurable threshold.

- Documents: AES-256-GCM envelope encryption, per-document DEKs wrapped by a KEK behind a ``KekProvider`` interface. UUID-based opaque storage paths. KEK rotation re-wraps DEKs without touching ciphertext.
- Electron: `nodeIntegration` false, `contextIsolation` true, `sandbox` true, `webSecurity` true, signed auto-updates, DevTools off in production builds.
- Database connects with the ``licensing_app`` role (least privilege). ``application_audit`` is INSERT + SELECT only; a BEFORE UPDATE OR DELETE trigger raises; the hash chain is the third defence layer.

8. Concurrency and audit integrity

8.1 Optimistic locking on applications

Every state-changing service method opens a transaction, SELECTs the application FOR UPDATE, validates the transition through the pure state machine, then UPDATEs ``state`` and ``row_version + 1`` with a ``WHERE row_version = $expected`` clause. If the count is zero, another writer beat us and we return ``409 VERSION_CONFLICT``. The audit row is written inside the same transaction.

8.2 Audit hash chain

Each audit entry stores ``entry_hash = SHA-256(canonical(entry) || previous_hash)``. Canonicalisation follows RFC 8785 (JCS): keys sorted, NFC-normalised strings, integer-only numbers, RFC 3339 UTC at millisecond precision, null fields omitted. The genesis `previous_hash` is 32 zero bytes. A standalone verifier (``scripts/verify-audit-chain.ts``) walks every chain and reports the first divergence per application.

8.3 Tamper resistance, in depth

- The running application has no privilege to UPDATE or DELETE the audit table.
- A trigger on ``application_audit`` raises on UPDATE, DELETE, TRUNCATE; bypassing it requires disabling the trigger, which is itself an auditable database event.
- The hash chain makes any tamper detectable retroactively even if a higher-privileged role circumvents both prior layers.
- Daily signed Merkle snapshots are produced; the WORM destination and the RFC 3161 TSA hook are deferred and listed under known limitations.

9. SLA tracking

Four clocks per application, evaluated by a 15-minute cron. Each clock has a `started_at`, a `due_at` computed from the regulation, a `warned_at` marker (set at 80% elapsed), a `breached_at` marker (set at `due_at`), and a `closed_at` marker (set when satisfied). Notifications fire at warn and breach; audit rows record both.

- COMPLETENESS: starts on submission; closes on letter-of-information; 10 working days.

- ASSESSMENT: starts on COMPLETE letter; closes on decision; 1 month.
- ABSOLUTE_DECISION: starts on COMPLETE letter; closes on decision; 3 months. Breach is the highest-severity alert.
- LICENCE_FEE: starts on grant decision; closes on fee proof upload; 1 month.

The SLA panel on each application surface renders a coloured progress bar per clock: green $\leq 80\%$, amber $> 80\%$, red breached. A banner appears above the tab group when any clock is breached and open.

10. Operational concerns

- Structured JSON logs (Pino) with correlation IDs on every line. Sensitive fields redacted before emit.
- `GET /healthz` returns liveness + database readiness.
- Prometheus-compatible `/metrics`: HTTP latency histograms, login failure counters, refresh-token reuse counter, audit chain verifier health, SLA breach counters.
- Daily encrypted base backups; continuous WAL archiving. Quarterly restore drill exercises the chain verifier against the restored database.
- Configuration is loaded from the environment with Zod validation; the API refuses to boot if any required value is missing.

11. Known limitations and deferred scope

Honest accounting of what this submission does not cover and the reasoning. The seams are in place; nothing here closes a future door.

Item	Status	Reason / planned remediation
Decisions table on the API	Frontend models the four outcomes; the API persists structured payloads in the decision justification text in this build.	Migration to a dedicated `decisions` table with type enum, conditions JSONB, allowed_activities, reasons JSONB is the next backend change.
Shareholder fit-and-proper attestation	UI present; backend lacks a dedicated `fit_and_proper_status` column on `significant_shareholders` (senior managers already have it).	Single migration plus a reviewer-only PATCH endpoint.
Permitted activities lookup	Free-text input on the limited-licence decision tab.	Planned `GET /applications/:id/permitted-activities` returning the activity set for the chosen category.
Post-grant workflow	State machine ends at APPROVED.	Reserve POST_GRANT_PENDING / PRE_OPENING_INSPECTION / LICENCE_ACTIVE states; implement once the operational policy is confirmed with the regulator.

Item	Status	Reason / planned remediation
Representative office	Out of scope.	Data model accommodates `application_kind` enum; separate workflow to be implemented as v2 feature.
Email and SMS notifications	In-app only.	Notification subsystem is interface-shaped; SMTP transport adds in the same code shape.
Document storage on object lock	Encrypted local disk in this build.	DocumentStorage interface; production binding to S3 with object lock is a single class swap.
KMS-backed signing keys	Loaded from environment via `KeyProvider`.	Same interface binds to AWS KMS / HashiCorp Vault in production.
WORM destination for audit snapshots	Daily signed snapshots written to a configured local directory.	Provision object-locked storage; same signer; no schema change needed.
Annual supervision-fee workflow	Schema seam only.	Out of scope; belongs to a separate supervisory portal.

12. How to run and test

12.1 Prerequisites

- Node 20 LTS.
- Docker (for local Postgres).

12.2 First-time setup

```
git clone <repo> licensing-portal && cd licensing-portal
cd apps/api && cp .env.example .env
docker compose up -d postgres
npm install    # at the monorepo root; workspaces install both apps
npm run api:migrate
npm run api:seed # prints local seed credentials at the end
npm run api:dev  # API on http://localhost:3000; OpenAPI at /api/docs
npm run web:dev  # in another terminal; web on http://localhost:4200
```

12.3 Test suite

```
npm run api:test      # backend unit + integration with coverage
npm run web:test      # frontend unit + component
npm run verify:audit-chain # walks every audit chain and reports divergences
Coverage gates: 90% line and branch on the repo; 95% line on auth, applications, and audit modules.
```

13. Architectural decisions (one-line summaries)

- ADR-0001: NestJS + Fastify + Prisma. Strong DI, typed validation, OpenAPI from decorators.
- ADR-0002: RS256 JWT with `kid` rotation. Asymmetric only.

- ADR-0003: Optimistic locking via `row\_version` on applications. Deterministic 409 on conflict.
- ADR-0004: Append-only audit at three layers (DB role, trigger, hash chain) with RFC 8785 canonicalisation.
- ADR-0005: API types are generated from the API's OpenAPI document and consumed by hand-written web services. No full-client generation.
- ADR-0006: Document versions are atomic per (application, slot, owner). Unique index plus `MAX(version)+1` inside a FOR UPDATE transaction.
- ADR-0007: SLA clocks are first-class entities, evaluated by a 15-minute cron. Notifications + audit on warn and breach.

14. Evidence map (non-negotiables → tests)

Requirement	Implementation	Test
Reviewer cannot also approve	SeparationOfDutiesGuard + access-policy	access-policy.separation.spec.ts
Illegal transitions rejected at API	state-machine.ts + applications.service.ts	applications.transition.spec.ts
Concurrent transitions deterministic	Optimistic locking in applications.service	applications.concurrency.spec.ts (50 iterations)
Audit append-only	DB role grants + trigger	audit.append-only.spec.ts
Audit hash chain	canonicaliser.ts + audit.service	canonicaliser.spec.ts + audit.chain.spec.ts
Document size cap	Streaming multipart guard	documents.size-cap.spec.ts
Document versioning under contention	unique (application_id, slot, owner, version) + transaction	documents.versioning.spec.ts
403 not 404 on authz	Global exception filter	error-codes.spec.ts
No stack traces in responses	Global exception filter	error-shape.spec.ts
Refresh-token rotation + reuse detection	refresh-tokens.service	refresh-token-rotation.spec.ts